JavaScript is disabled on your browser.

**Java SE 27 & JDK 27**

# Class Locale

java.lang.Object
    java.util.Locale

**All Implemented Interfaces:**

`Serializable`, `Cloneable`

---

```
public final class Locale
extends Object
implements Cloneable, Serializable
```

A `Locale` represents a specific geographical, political, or cultural region. An API that requires a `Locale` to perform its task is *locale-sensitive* and uses the `Locale` to tailor information for the user. These *locale-sensitive* APIs are principally in the *java.text* and *java.util* packages. For example, displaying a number is a *locale-sensitive* operation— the number should be formatted according to the customs and conventions of the user's native country, region, or culture.

The `Locale` class implements IETF BCP 47 which contains RFC 4647 "Matching of Language Tags" and RFC 5646 "Tags for Identifying Languages" with support for the LDML (UTS#35, "Unicode Locale Data Markup Language") BCP 47-compatible extensions for locale data exchange. Each `Locale` is associated with locale data which is provided by the Java runtime environment or any deployed `LocaleServiceProvider` implementations. The locale data provided by the Java runtime environment may vary by release.

## Locale Composition

A `Locale` is composed of the bolded fields described below; note that a `Locale` need not have all such fields. For example, `Locale.ENGLISH` is only comprised of the *language* field. In contrast, a `Locale` such as the one returned by `Locale.forLanguageTag("en-Latn-US-POSIX-u-nu-latn")` would be comprised of all the fields below. This particular `Locale` would represent English in the United States using the Latin script and numerics for use in POSIX environments.

`Locale` implements IETF BCP 47 and any deviations should be observed by the comments prefixed by "*BCP 47 deviation:*". RFC 5646 combines subtags from various ISO (639, 3166, 15924) standards which are also included in the composition of `Locale`. Additionally, the full list of valid codes for each field can be found in the IANA Language Subtag Registry (e.g. search for "Type: region").

**language**

ISO 639 alpha-2/alpha-3 language code or a registered language subtag up to 8 alpha letters (for future enhancements). When a language has both an alpha-2 code and an alpha-3 code, the alpha-2 code must be used.
*Case convention:* `language` is case insensitive, but `Locale` always canonicalizes to lower case.
*Syntax:* Well-formed `language` values have the form `[a-zA-Z]{2,8}`.
*BCP 47 deviation:* `Locale` does not retain the extlang subtag. This is because three-letter language codes are preferred over extlang subtags. When a `Locale` is created from a language tag containing an extlang subtag, the first extlang subtag is interpreted as the *language* field. The primary language subtag and any subsequent extlang subtags are ignored.
*Example:* "en" (English), "ja" (Japanese), "kok" (Konkani)

**script**

ISO 15924 alpha-4 script code.
*Case convention:* `script` is case insensitive, but `Locale` always canonicalizes to title case (the first letter is upper case and the rest of the letters are lower case).
*Syntax:* Well-formed `script` values have the form `[a-zA-Z]{4}`
*Example:* "Latn" (Latin), "Cyrl" (Cyrillic)

**country (region)**

ISO 3166 alpha-2 country code or UN M.49 numeric-3 area code.
*Case convention:* `country (region)` is case insensitive, but `Locale` always canonicalizes to upper case.
*Syntax:* Well-formed `country (region)` values have the form `[a-zA-Z]{2} | [0-9]{3}`
*Example:* "US" (United States), "FR" (France), "029" (Caribbean)

**variant**

Any arbitrary value used to indicate a variation of a `Locale`. When multiple variants exist, they should be separated by `('_'|'-')`. Variants of higher importance should precede the others.
*BCP 47 deviation:* BCP 47 subtags are strictly used to indicate additional variations that define a language or its dialects that are not covered by any combinations of language, script and region subtags. However, the variant field in `Locale` has historically been used for any kind of variation, not just language variations. For example, some supported variants available in Java SE Runtime Environments indicate alternative cultural behaviors such as calendar type or number script. In BCP 47, this kind of information which does not identify the language, is supported by extension subtags or private use subtags.
*Case convention:* `variant` is case sensitive. BCP 47 deviation: BCP 47 treats the variant field as case insensitive.
*Syntax:* Well-formed `variant` values have the form `SUBTAG (('_'|'-') SUBTAG)*` where `SUBTAG = [0-9][0-9a-zA-Z]{3} | [0-9a-zA-Z]{5,8}`.

*BCP 47 deviation:* BCP 47 only uses hyphen ('-') as a delimiter and APIs provided by `Locale` which accept BCP 47 language tags expect as such. However, for backwards compatibility, `Locale.Builder.setVariant(String)` also accepts underscore ('_'). `of(String, String, String)` accepts only underscore ('_').
*Example:* "polyton" (Polytonic Greek), "POSIX"

**extensions**

A map from single character keys to string values, indicating extensions apart from language identification.
*BCP 47 deviation:* The `extensions` in `Locale` implement the semantics and syntax of BCP 47 extension subtags *and* private use subtags. The `extensions` field cannot have empty values.
*Case convention:* `extensions` are case insensitive, but `Locale` canonicalizes all extension keys and values to lower case.
*Syntax:* Well-formed keys are single characters from the set `[0-9a-zA-Z]`. Well-formed values have the form `SUBTAG ('-' SUBTAG)*` where for the key 'x' `SUBTAG = [0-9a-zA-Z]{1,8}` and for other keys `SUBTAG = [0-9a-zA-Z]{2,8}` (that is, 'x' allows single-character subtags).
*Example:* key="u"/value="ca-japanese" (Japanese Calendar), key="x"/value="java-1-7"

**BCP 47 deviation:** BCP47 defines the following two levels of conformance, "valid" and "well-formed". A valid tag requires that it is well-formed, its subtag values are registered in the IANA Language Subtag Registry, and it does not contain duplicate variant or extension singleton subtags. The `Locale` class does not enforce that subtags are registered in the Subtag Registry. `Locale.Builder` only checks if an individual field satisfies the syntactic requirement (is well-formed). When passed duplicate variants, `Builder` accepts and includes them. When passed duplicate extension singletons, `Builder` accepts but ignores the duplicate key and its associated value. Conversely, `Locale::of` and its overloads do not check if the input is well-formed at all.

## Unicode BCP 47 U Extension

UTS#35, "Unicode Locale Data Markup Language" defines the Unicode BCP 47 U Extension, an extension based on RFC 6067, which describes optional attributes and keywords to override or refine the default behavior associated with a locale. A keyword is represented by a pair of key and type. For example, "nu-thai" indicates that Thai local digits (value:"thai") should be used for formatting numbers (key:"nu").

The keywords are mapped to a BCP 47 extension value using the extension key 'u' (`UNICODE_LOCALE_EXTENSION`). The above example, "nu-thai", becomes the extension "u-nu-thai".

Thus, when a `Locale` object contains Unicode locale attributes and keywords, `getExtension(UNICODE_LOCALE_EXTENSION)` will return a String representing this information, for example, "nu-thai". The `Locale` class also provides `getUnicodeLocaleAttributes()`, `getUnicodeLocaleKeys()`, and `getUnicodeLocaleType(String)` which provides access to the Unicode locale attributes and key/type pairs directly. When represented as a string, the Unicode Locale Extension lists attributes alphabetically, followed by key/type sequences with keys listed alphabetically (the order of subtags comprising a key's type is fixed when the type is defined)

A well-formed locale key has the form `[0-9a-zA-Z]{2}`. A well-formed locale type has the form `"" | [0-9a-zA-Z]{3,8} ('-' [0-9a-zA-Z]{3,8})*` (it can be empty, or a series of subtags 3-8 alphanums in length). A well-formed locale attribute has the form `[0-9a-zA-Z]{3,8}` (it is a single subtag with the same form as a locale type subtag). Duplicate locale attributes as well as locale keys do not convey meaning. For methods in `Locale` and `Locale.Builder` that accept extensions, occurrences of duplicate locale attributes as well as locale keys and their associated type are accepted but ignored.

The Unicode locale extension specifies optional behavior in locale-sensitive services. Although the LDML specification defines various keys and values, actual locale-sensitive service implementations in a Java Runtime Environment might not support any particular Unicode locale attributes or key/type pairs.

## Default Locale

The default Locale is provided for any *locale-sensitive* methods if no `Locale` is explicitly specified as an argument, such as `NumberFormat.getInstance()`. The default Locale is determined at startup of the Java runtime and established in the following three phases:

1. The locale-related system properties listed below are established from the host environment. Some system properties (except for `user.language`) may not have values from the host environment.

| **Locale-related System Properties Key** | **Description** |
|---|---|
| `user.language` | `language` for the default Locale, such as "en" (English) |
| `user.script` | `script` for the default Locale, such as "Latn" (Latin) |
| `user.country` | `country` for the default Locale, such as "US" (United States) |
| `user.variant` | `variant` for the default Locale, such as "POSIX" |

| Locale-related System Properties Key | Description |
| --- | --- |
| `user.extensions` | `extensions` for the default Locale, such as "u-ca-japanese" (Japanese Calendar) |

2. The values of these system properties can be overridden by values designated at startup time. If the overriding value of the `user.extensions` property is unparsable, it is ignored. The overriding values of other properties are not checked for syntax or validity and are used directly in the default Locale. (Typically, system property values can be provided using the `-D` command-line option of a launcher. For example, specifying `-Duser.extensions=foobarbaz` results in a default Locale with no extensions, while specifying `-Duser.language=foobarbaz` results in a default Locale whose language is "foobarbaz".)
3. The default `Locale` instance is constructed from the values of these system properties.

Altering the system property values with `System.setProperties(Properties)`/ `System.setProperty(String, String)` has no effect on the default Locale.

Once the default Locale is established, applications can query the default Locale with `getDefault()` and change it with `setDefault(Locale)`. If the default Locale is changed with `setDefault(Locale)`, the corresponding system properties are not altered. It is not recommended that applications read these system properties and parse or interpret them as their values may be out of date.

### Locale Category

There are finer-grained default Locales specific for each `Locale.Category`. These category specific default Locales can be queried by `getDefault(Category)`, and set by `setDefault(Category, Locale)`. Construction of these category specific default Locales are determined by the corresponding system properties, which consist of the base system properties as listed above, suffixed by either `".display"` or `".format"` depending on the category. For example, the value of the `user.language.display` system property will be used in the `language` part of the default Locale for the `Locale.Category.DISPLAY` category. In the absence of category specific system properties, the "category-less" system properties are used, such as `user.language` in the previous example.

## Obtaining a Locale

There are several ways to obtain a `Locale` object. It is advised against using the deprecated `Locale` constructors.

**Locale Constants**

A number of convenient constants are provided that return `Locale` objects for commonly used locales. For example, `Locale.US` is the `Locale` object for the United States.

**Factory Methods**

`Locale::of` and its overloads obtain a `Locale` object from the given `language`, `country`, and/or `variant`. `forLanguageTag(String)` obtains a `Locale` object for a well-formed BCP 47 language tag.

**Builder**

`Locale.Builder` is used to construct a `Locale` object that conforms to BCP 47 syntax. Use a builder to enforce syntactic restrictions on the input.

The following invocations produce Locale objects that are all equivalent:

```
Locale.US;
Locale.of("en", "US");
Locale.forLanguageTag("en-US");
new Locale.Builder().setLanguage("en").setRegion("US").build();
```

## Usage Examples

Once a `Locale` is obtained, it can be queried for information about itself. For example, use `getCountry()` to get the country (or region) code and `getLanguage()` to get the language. `getDisplayCountry()` can be used to get the name of the country suitable for displaying to the user. Similarly, use `getDisplayLanguage()` to get the name of the language suitable for displaying to the user. The `getDisplayXXX` methods are themselves *locale-sensitive* and have two variants; one with an explicit locale parameter, and one without. The latter uses the default `DISPLAY` locale, so the following are equivalent :

```
Locale.getDefault().getDisplayCountry();
Locale.getDefault().getDisplayCountry(Locale.getDefault(Locale.Category.DISPL
```

The Java Platform provides a number of classes that perform locale-sensitive operations. For example, the `NumberFormat` class formats numbers, currency, and percentages in a *locale-sensitive* manner. Classes such as `NumberFormat` have several factory methods for creating a default object of that type. These methods generally have two variants; one with an explicit locale parameter, and one without. The latter uses the default `FORMAT` locale, so the following are equivalent :

```
    NumberFormat.getCurrencyInstance();
    NumberFormat.getCurrencyInstance(Locale.getDefault(Locale.Category.FORMAT));
```

The following example demonstrates *locale-sensitive* currency and date related operations under different locales :

```
    var number = 1000;
    NumberFormat.getCurrencyInstance(Locale.US).format(number); // returns "$1,00
    NumberFormat.getCurrencyInstance(Locale.JAPAN).format(number); // returns "¥1
    var date = LocalDate.of(2024, 1, 1);
    DateTimeFormatter.ofLocalizedDate(FormatStyle.LONG).localizedBy(Locale.US).fo
    DateTimeFormatter.ofLocalizedDate(FormatStyle.LONG).localizedBy(Locale.JAPAN)
```

## Locale Matching

If an application is internationalized and provides localized resources for multiple locales, it sometimes needs to find one or more locales (or language tags) which meet each user's specific preferences. Note that the term "*language tag*" is used interchangeably with "locale" in the following locale matching documentation.

In order to match a user's preferred locales to a set of language tags, RFC 4647 Matching of Language Tags defines two mechanisms: filtering and lookup. *Filtering* is used to get all matching locales, whereas *lookup* is to select the best matching locale. Matching is done case-insensitively. These matching mechanisms are described in the following sections.

A user's preference is called a *Language Priority List* and is expressed as a list of language ranges. There are syntactically two types of language ranges: basic and extended. See `Locale.LanguageRange` for details.

### Filtering

The filtering operation returns all matching language tags. It is defined in RFC 4647 as follows: "In filtering, each language range represents the least specific language tag (that is, the language tag with the fewest number of subtags) that is an acceptable match. All the language tags in the matching set of tags will have an equal or greater number of subtags than the language range. Every non-wildcard subtag in the language range will appear in every one of the matching language tags."

There are two types of filtering: filtering for basic language ranges (called "basic filtering") and filtering for extended language ranges (called "extended filtering"). They may return different results by what kind of language ranges

are included in the given Language Priority List. `Locale.FilteringMode` is a parameter to specify how filtering should be done.

### Lookup

The lookup operation returns the best matching language tags. It is defined in RFC 4647 as follows: "By contrast with filtering, each language range represents the most specific tag that is an acceptable match. The first matching tag found, according to the user's priority, is considered the closest match and is the item returned."

For example, if a Language Priority List consists of two language ranges, `"zh-Hant-TW"` and `"en-US"`, in prioritized order, lookup method progressively searches the language tags below in order to find the best matching language tag.

```
1. zh-Hant-TW
2. zh-Hant
3. zh
4. en-US
5. en
```

If there is a language tag which matches completely to a language range above, the language tag is returned.

`"*"` is the special language range, and it is ignored in lookup.

If multiple language tags match as a result of the subtag `'*'` included in a language range, the first matching language tag returned by an `Iterator` over a `Collection` of language tags is treated as the best matching one.

### Serialization

During serialization, writeObject writes all fields to the output stream, including extensions.

During deserialization, readResolve adds extensions as described in Special Cases, only for the two cases th_TH_TH and ja_JP_JP.

#### Implementation Note:

## Compatibility

The following commentary is provided for apps that want to ensure interoperability with older releases of `Locale` provided by the reference implementation.

## Locale Behavior

In order to maintain compatibility, Locale's (deprecated) constructors, `of(String, String, String)`, and its overloads retain their behavior prior to the Java Runtime Environment version 1.7. That is, a length constraint is not imposed on any of the input parameters. Similarly, the same preservation of past behavior is largely true for the `toString()` method. Apps that previously parsed the output of `toString()` into language, country, and variant fields can continue to do so (although this is strongly discouraged). A caveat is that the variant field will have additional information in it if script or extensions are present.

In addition, BCP 47 imposes syntax restrictions that are not imposed by Locale's constructors. This means that conversions between some Locales and BCP 47 language tags cannot be made without losing information. Thus `toLanguageTag()` cannot represent the state of locales whose language, country, or variant do not conform to BCP 47.

Because of these issues, it is recommended that apps migrate away from constructing non-conforming locales and use the `forLanguageTag(String)` and `Locale.Builder` APIs instead. Apps desiring a string representation of the complete locale can then always rely on `toLanguageTag()` for this purpose.

## Special cases

For compatibility reasons, two non-conforming locales are treated as special cases. These are **`ja_JP_JP`** and **`th_TH_TH`**. These are ill-formed in BCP 47 since the variants are too short. To ease migration to BCP 47, these are treated specially during construction. These two cases (and only these) cause a constructor to generate an extension, all other values behave exactly as they did prior to Java 7.

Java has used `ja_JP_JP` to represent Japanese as used in Japan together with the Japanese Imperial calendar. This is now representable using a Unicode locale extension, by specifying the Unicode locale key `ca` (for "calendar") and type `japanese`. When the Locale constructor is called with the arguments "ja", "JP", "JP", the extension "u-ca-japanese" is automatically added.

Java has used `th_TH_TH` to represent Thai as used in Thailand together with Thai digits. This is also now representable using a Unicode locale extension, by specifying the Unicode locale key `nu` (for "number") and value `thai`. When the Locale constructor is called with the arguments "th", "TH", "TH", the extension "u-nu-thai" is automatically added.

### Legacy language codes

For compatibility, a `Locale` created with one of the three obsolete language codes, `iw`, `ji`, or `in`, will map the language to its modern equivalent, `he`, `yi`, or `id`, respectively.

The default resource bundle lookup mechanism also implements this mapping, so that resources can be named using either convention, see `ResourceBundle.Control`.

**Since:**

1.1

**External Specifications**

IETF BCP 47,
RFC 4647: Matching of Language Tags,
RFC 5646: Tags for Identifying Languages,
RFC 6067: BCP 47 Extension U,
Unicode Locale Data Markup Language (LDML)

**See Also:**

`Locale.Builder`, `ResourceBundle`, `Format`, `NumberFormat`, `Collator`, Serialized Form

## *Nested Class Summary*

**Nested Classes**

| Modifier and Type | Class | Description |
|---|---|---|
| `static final class` | **`Locale.Builder`** | `Builder` is used to build instances of `Locale` from values configured by the setters. |
| `static enum` | **`Locale.Category`** | Enum for locale categories. |
| `static enum` | **`Locale.FilteringMode`** | This enum provides constants to select a filtering mode for locale matching. |
| `static enum` | **`Locale.IsoCountryCode`** | Enum for specifying the type defined in ISO 3166. |
| `static final class` | **`Locale.LanguageRange`** | |

This class expresses a *Language Range* defined in RFC 4647 Matching of Language Tags.

## *Field Summary*

**Fields**

| Modifier and Type | Field | Description |
|---|---|---|
| static final Locale | **CANADA** | Useful constant for country. |
| static final Locale | **CANADA_FRENCH** | Useful constant for country. |
| static final Locale | **CHINA** | Useful constant for country. |
| static final Locale | **CHINESE** | Useful constant for language. |
| static final Locale | **ENGLISH** | Useful constant for language. |
| static final Locale | **FRANCE** | Useful constant for country. |
| static final Locale | **FRENCH** | Useful constant for language. |
| static final Locale | **GERMAN** | Useful constant for language. |
| static final Locale | **GERMANY** | Useful constant for country. |
| static final Locale | **ITALIAN** | Useful constant for language. |
| static final Locale | **ITALY** | Useful constant for country. |
| static final Locale | **JAPAN** | Useful constant for country. |

| | |
|---|---|
| static final Locale **JAPANESE** | Useful constant for language. |
| static final Locale **KOREA** | Useful constant for country. |
| static final Locale **KOREAN** | Useful constant for language. |
| static final Locale **PRC** | Useful constant for country. |
| static final char **PRIVATE_USE_EXTENSION** | The key for the private use extension ('x'). |
| static final Locale **ROOT** | Useful constant for the root locale. |
| static final Locale **SIMPLIFIED_CHINESE** | Useful constant for language. |
| static final Locale **TAIWAN** | Useful constant for country. |
| static final Locale **TRADITIONAL_CHINESE** | Useful constant for language. |
| static final Locale **UK** | Useful constant for country. |
| static final char **UNICODE_LOCALE_EXTENSION** | The key for Unicode locale extension ('u'). |
| static final Locale **US** | Useful constant for country. |

## *Constructor Summary*

**Constructors**

| Constructor | Description |
|---|---|
| **Locale**(String language) | **Deprecated.** Locale constructors have been deprecated. |
| | **Deprecated.** |

| | |
|---|---|
| **Locale**(String language, String country) | Locale constructors have been deprecated. |
| **Locale**(String language, String country, String variant) | **Deprecated.** Locale constructors have been deprecated. |

## *Method Summary*

All Methods | Static Methods

| Modifier and Type | Method | Description |
|---|---|---|
| static Stream<Locale> | **availableLocales**() | Returns a stream of available locales. |
| static String | **caseFoldLanguageTag**(String languageTag) | Returns a case folded IETF BCP 47 language tag. |
| Object | **clone**() | Overrides Cloneable. |
| boolean | **equals**(Object obj) | Returns true if this Locale is equal to another object. |
| static List<Locale> | **filter**(List<Locale.LanguageRan Collection<Locale> local | Returns a list of matching Locale instances using the filtering mechanism defined in RFC 4647. |
| static List<Locale> | **filter**(List<Locale.LanguageRan Collection<Locale> local Locale.FilteringMode mod | Returns a list of matching Locale instances using the filtering mechanism defined in RFC 4647. |
| static List<String> | **filterTags**(List<Locale.LanguageRan Collection<String> tags) | Returns a list of matching languages tags using the basic filtering mechanism defined in RFC 4647. |
| static List<String> | | |

| | | |
|---|---|---|
| | **filterTags**(List<Locale.LanguageRan Collection<String> tags, Locale.FilteringMode mod | Returns a list of matching languages tags using the basic filtering mechanism defined in RFC 4647. |
| static Locale | **forLanguageTag**(String languageTag) | Returns a locale for the specified IETF BCP 47 language tag string. |
| static Locale[] | **getAvailableLocales**() | Returns an array of available locales. |
| String | **getCountry**() | Returns the country/ region code for this locale, which should either be the empty string, an uppercase ISO 3166 2-letter code, or a UN M.49 3-digit code. |
| static Locale | **getDefault**() | Gets the current value of the default locale for this instance of the Java Virtual Machine. |
| static Locale | **getDefault**(Locale.Category categor | Gets the current value of the default locale for the specified Category for this instance of the Java Virtual Machine. |
| String | **getDisplayCountry**() | Returns a name for this locale's country that is appropriate for display to the user. |
| String | **getDisplayCountry**(Locale inLocale) | Returns a name for this locale's country that is appropriate for display to the user. |
| String | **getDisplayLanguage**() | Returns a name for this locale's language that is appropriate for display to the user. |

| | | |
|---|---|---|
| String | **getDisplayLanguage**(Locale inLocale) | Returns a name for this locale's language that is appropriate for display to the user. |
| String | **getDisplayName**() | Returns a name for this locale that is appropriate for display to the user. |
| String | **getDisplayName**(Locale inLocale) | Returns a name for this locale that is appropriate for display to the user. |
| String | **getDisplayScript**() | Returns a name for this locale's script that is appropriate for display to the user. |
| String | **getDisplayScript**(Locale inLocale) | Returns a name for this locale's script that is appropriate for display to the user. |
| String | **getDisplayVariant**() | Returns a name for this locale's variant code that is appropriate for display to the user. |
| String | **getDisplayVariant**(Locale inLocale) | Returns a name for this locale's variant code that is appropriate for display to the user. |
| String | **getExtension**(char key) | Returns the extension (or private use) value associated with the specified key, or null if there is no extension associated with the key. |
| Set<Character> | **getExtensionKeys**() | Returns the set of extension keys associated with this locale, or the empty set if it has no extensions. |

| | | |
|---|---|---|
| String | **getISO3Country**() | Returns a three-letter abbreviation of this locale's country. |
| String | **getISO3Language**() | Returns a three-letter abbreviation of this locale's language. |
| static String[] | **getISOCountries**() | Returns a list of all 2-letter country codes defined in ISO 3166. |
| static Set<String> | **getISOCountries** (Locale.IsoCountryCode t | Returns an unmodifiable Set of ISO3166 country codes for the specified type. |
| static String[] | **getISOLanguages**() | Returns a list of all 2-letter language codes defined in ISO 639. |
| String | **getLanguage**() | Returns the language code of this Locale. |
| String | **getScript**() | Returns the script for this locale, which should either be the empty string or an ISO 15924 4-letter script code. |
| Set<String> | **getUnicodeLocaleAttribut** | Returns the set of unicode locale attributes associated with this locale, or the empty set if it has no attributes. |
| Set<String> | **getUnicodeLocaleKeys**() | Returns the set of Unicode locale keys defined by this locale, or the empty set if this locale has none. |
| String | **getUnicodeLocaleType** (String key) | Returns the Unicode locale type associated with the specified |

| | | |
|---|---|---|
| | | Unicode locale key for this locale. |
| String | **getVariant**() | Returns the variant code for this locale. |
| boolean | **hasExtensions**() | Returns true if this Locale has any extensions. |
| int | **hashCode**() | Override hashCode. |
| static Locale | **lookup**(List<Locale.LanguageRan Collection<Locale> local | Returns a Locale instance for the best-matching language tag using the lookup mechanism defined in RFC 4647. |
| static String | **lookupTag**(List<Locale.LanguageRan Collection<String> tags) | Returns the best-matching language tag using the lookup mechanism defined in RFC 4647. |
| static Locale | **of**(String language) | Obtains a locale from a language code. |
| static Locale | **of**(String language, String country) | Obtains a locale from language and country. |
| static Locale | **of**(String language, String country, String variant) | Obtains a locale from language, country and variant. |
| static void | **setDefault**(Locale newLocale) | Sets the default locale for this instance of the Java Virtual Machine. |
| static void | **setDefault**(Locale.Category categor Locale newLocale) | Sets the default locale for the specified Category for this instance of the Java Virtual Machine. |

| | | |
|---|---|---|
| Locale | **stripExtensions**() | Returns a copy of this Locale with no extensions. |
| String | **toLanguageTag**() | Returns a well-formed IETF BCP 47 language tag representing this locale. |
| final String | **toString**() | Returns a string representation of this Locale object, consisting of language, country, variant, script, and extensions as below: language + "_" + country + "_" + (variant + "_#" \| "#") + script + "_" + extensions Language is always lower case, country is always upper case, script is always title case, and extensions are always lower case. |

**Methods declared in class Object**

finalize, getClass, notify, notifyAll, wait, wait, wait

## *Field Details*

### ENGLISH

```
public static final Locale ENGLISH
```

Useful constant for language.

### FRENCH

`public static final Locale FRENCH`

Useful constant for language.

### GERMAN

`public static final Locale GERMAN`

Useful constant for language.

### ITALIAN

`public static final Locale ITALIAN`

Useful constant for language.

### JAPANESE

`public static final Locale JAPANESE`

Useful constant for language.

### KOREAN

`public static final Locale KOREAN`

Useful constant for language.

### CHINESE

`public static final Locale CHINESE`

Useful constant for language.

### SIMPLIFIED_CHINESE

`public static final Locale SIMPLIFIED_CHINESE`

Useful constant for language.

### TRADITIONAL_CHINESE

`public static final Locale TRADITIONAL_CHINESE`

Useful constant for language.

### FRANCE

```
public static final Locale FRANCE
```

Useful constant for country.

### GERMANY

```
public static final Locale GERMANY
```

Useful constant for country.

### ITALY

```
public static final Locale ITALY
```

Useful constant for country.

### JAPAN

```
public static final Locale JAPAN
```

Useful constant for country.

### KOREA

```
public static final Locale KOREA
```

Useful constant for country.

### UK

```
public static final Locale UK
```

Useful constant for country.

### US

```
public static final Locale US
```

Useful constant for country.

### CANADA

```
public static final Locale CANADA
```

Useful constant for country.

### CANADA_FRENCH

`public static final Locale CANADA_FRENCH`

Useful constant for country.

### ROOT

`public static final Locale ROOT`

Useful constant for the root locale. The root locale is the locale whose language, country, and variant are empty ("") strings. This is regarded as the base locale of all locales, and is used as the language/country neutral locale for the locale sensitive operations.

**Since:**

1.6

### CHINA

`public static final Locale CHINA`

Useful constant for country.

### PRC

`public static final Locale PRC`

Useful constant for country.

### TAIWAN

`public static final Locale TAIWAN`

Useful constant for country.

### PRIVATE_USE_EXTENSION

`public static final char PRIVATE_USE_EXTENSION`

The key for the private use extension ('x').

**Since:**

1.7

**See Also:**

`getExtension(char)`,
`Locale.Builder.setExtension(char, String)`,
Constant Field Values

### UNICODE_LOCALE_EXTENSION

```
public static final char UNICODE_LOCALE_EXTENSION
```

The key for Unicode locale extension ('u').

**Since:**

1.7

**See Also:**

`getExtension(char)`,
`Locale.Builder.setExtension(char, String)`,
Constant Field Values

## *Constructor Details*

### Locale

```
@Deprecated(since="19")
public Locale(String language,
              String country,
              String variant)
```

**Deprecated.**
*Locale constructors have been deprecated. See Obtaining a Locale for other options.*

Construct a locale from language, country and variant. This constructor normalizes the language value to lowercase and the country value to uppercase.

**Implementation Note:**

- Obsolete ISO 639 codes ("iw", "ji", and "in") are mapped to their current forms. See Legacy language codes for more information.
- For backward compatibility reasons, this constructor does not make any syntactic checks on the input.
- The two cases ("ja", "JP", "JP") and ("th", "TH", "TH") are handled specially, see Special Cases for more information.

**Parameters:**

`language` - An ISO 639 alpha-2 or alpha-3 language code, or a language subtag up to 8 characters in length. See the `Locale` class description about valid language values.

`country` - An ISO 3166 alpha-2 country code or a UN M.49 numeric-3 area code. See the `Locale` class description about valid country values.

`variant` - Any arbitrary value used to indicate a variation of a `Locale`. See the `Locale` class description for the details.

**Throws:**

`NullPointerException` - thrown if any argument is null.

## Locale

```
@Deprecated(since="19")
public Locale(String language,
              String country)
```

**Deprecated.**
*Locale constructors have been deprecated. See Obtaining a Locale for other options.*

Construct a locale from language and country. This constructor normalizes the language value to lowercase and the country value to uppercase.

**Implementation Note:**

- Obsolete ISO 639 codes ("iw", "ji", and "in") are mapped to their current forms. See Legacy language codes for more information.
- For backward compatibility reasons, this constructor does not make any syntactic checks on the input.

**Parameters:**

`language` - An ISO 639 alpha-2 or alpha-3 language code, or a language subtag up to 8 characters in length. See the `Locale` class description about valid language values.

`country` - An ISO 3166 alpha-2 country code or a UN M.49 numeric-3 area code. See the `Locale` class description about valid country values.

**Throws:**

`NullPointerException` - thrown if either argument is null.

## Locale

```
@Deprecated(since="19")
public Locale(String language)
```

**Deprecated.**
*Locale constructors have been deprecated. See Obtaining a Locale for other options.*

Construct a locale from a language code. This constructor normalizes the language value to lowercase.

**Implementation Note:**

- Obsolete ISO 639 codes ("iw", "ji", and "in") are mapped to their current forms. See Legacy language codes for more information.
- For backward compatibility reasons, this constructor does not make any syntactic checks on the input.

**Parameters:**

`language` - An ISO 639 alpha-2 or alpha-3 language code, or a language subtag up to 8 characters in length. See the `Locale` class description about valid language values.

**Throws:**

`NullPointerException` - thrown if argument is null.

**Since:**

1.4

## *Method Details*

### of

```
public static Locale of(String language,
                        String country,
                        String variant)
```

Obtains a locale from language, country and variant. This method normalizes the language value to lowercase and the country value to uppercase.

**Implementation Note:**

- This method does not make any syntactic checks on the input. Use `Locale.Builder` for full syntactic checks with BCP47.
- The two cases ("ja", "JP", "JP") and ("th", "TH", "TH") are handled specially, see Special Cases for more information.
- Obsolete ISO 639 codes ("iw", "ji", and "in") are mapped to their current forms. See Legacy language codes for more information.

**Parameters:**

`language` - A language code. See the `Locale` class description of language values.

`country` - A country code. See the `Locale` class description of country values.

`variant` - Any arbitrary value used to indicate a variation of a `Locale`. See the `Locale` class description of variant values.

**Returns:**

A `Locale` object

**Throws:**

`NullPointerException` - thrown if any argument is null.

**Since:**

19

## of

```
public static Locale of(String language,
                        String country)
```

Obtains a locale from language and country. This method normalizes the language value to lowercase and the country value to uppercase.

**Implementation Note:**

- This method does not make any syntactic checks on the input. Use `Locale.Builder` for full syntactic checks with BCP47.
- Obsolete ISO 639 codes ("iw", "ji", and "in") are mapped to their current forms. See Legacy language codes for more information.

**Parameters:**

`language` - A language code. See the `Locale` class description of language values.

`country` - A country code. See the `Locale` class description of country values.

**Returns:**

A `Locale` object

**Throws:**

`NullPointerException` - thrown if either argument is null.

**Since:**

19

## of

```
public static Locale of(String language)
```

Obtains a locale from a language code. This method normalizes the language value to lowercase.

**Implementation Note:**

- This method does not make any syntactic checks on the input. Use `Locale.Builder` for full syntactic checks with BCP47.
- Obsolete ISO 639 codes ("iw", "ji", and "in") are mapped to their current forms. See Legacy language codes for more information.

**Parameters:**

`language` - A language code. See the `Locale` class description of language values.

**Returns:**

A `Locale` object

**Throws:**

`NullPointerException` - thrown if argument is null.

**Since:**

19

## getDefault

```
public static Locale getDefault()
```

Gets the current value of the `default locale` for this instance of the Java Virtual Machine.

The Java Virtual Machine sets the default locale during startup based on the host environment. It is used by many locale-sensitive methods if no locale is explicitly specified. It can be changed using the `setDefault(Locale)` method.

**Returns:**

the default locale for this instance of the Java Virtual Machine

## getDefault

```
public static Locale getDefault(Locale.Category category)
```

Gets the current value of the `default locale` for the specified Category for this instance of the Java Virtual Machine.

The Java Virtual Machine sets the default locale during startup based on the host environment. It is used by many locale-sensitive methods if no locale is explicitly specified. It can be changed using the `setDefault(Locale.Category, Locale)` method.

**Parameters:**

`category` - the specified category to get the default locale

**Returns:**

the default locale for the specified Category for this instance of the Java Virtual Machine

**Throws:**

`NullPointerException` - if category is null

**Since:**

1.7

**See Also:**

`setDefault(Locale.Category, Locale)`

## setDefault

```
public static void setDefault(Locale newLocale)
```

Sets the `default locale` for this instance of the Java Virtual Machine. This does not affect the host locale.

The Java Virtual Machine sets the default locale during startup based on the host environment. It is used by many locale-sensitive methods if no locale is explicitly specified.

Since changing the default locale may affect many different areas of functionality, this method should only be used if the caller is prepared to reinitialize locale-sensitive code running within the same Java Virtual Machine.

By setting the default locale with this method, all of the default locales for each Category are also set to the specified default locale.

**Parameters:**

`newLocale` - the new default locale

**Throws:**

`NullPointerException` - if `newLocale` is null

## setDefault

```
public static void setDefault(Locale.Category category,
                              Locale newLocale)
```

Sets the `default locale` for the specified Category for this instance of the Java Virtual Machine. This does not affect the host locale.

The Java Virtual Machine sets the default locale during startup based on the host environment. It is used by many locale-sensitive methods if no locale is explicitly specified.

Since changing the default locale may affect many different areas of functionality, this method should only be used if the caller is prepared to reinitialize locale-sensitive code running within the same Java Virtual Machine.

**Parameters:**

`category` - the specified category to set the default locale

`newLocale` - the new default locale

**Throws:**

`NullPointerException` - if category and/or newLocale is null

**Since:**

1.7

**See Also:**

`getDefault(Locale.Category)`

## getAvailableLocales

```
public static Locale[] getAvailableLocales()
```

Returns an array of available locales. The returned array represents the union of locales supported by the Java runtime environment and by deployed `LocaleServiceProvider` implementations. At a minimum, the returned array must contain a `Locale` instance equal to `Locale.ROOT` and a `Locale` instance equal to `Locale.US`.

**Returns:**

an array of available locales

## availableLocales

```
public static Stream<Locale> availableLocales()
```

Returns a stream of available locales. The returned stream represents the union of locales supported by the Java runtime environment and by deployed `LocaleServiceProvider` implementations. At a minimum, the returned stream must contain a `Locale` instance equal to `Locale.ROOT` and a `Locale` instance equal to `Locale.US`.

**Implementation Note:**

Unlike `getAvailableLocales()`, this method does not create a defensive copy of the Locale array.

**Returns:**

a stream of available locales

**Since:**

21

## getISOCountries

```
public static String[] getISOCountries()
```

Returns a list of all 2-letter country codes defined in ISO 3166. Can be used to obtain Locales. This method is equivalent to

`getISOCountries(Locale.IsoCountryCode type)` with `type` `Locale.IsoCountryCode.PART1_ALPHA2`.

**Note:** The `Locale` class also supports other codes for country (region), such as 3-letter numeric UN M.49 area codes. Therefore, the list returned by this method does not contain ALL valid codes that can be used to obtain Locales.

Note that this method does not return obsolete 2-letter country codes. ISO3166-3 codes which designate country codes for those obsolete codes, can be retrieved from `getISOCountries(Locale.IsoCountryCode type)` with `type` `Locale.IsoCountryCode.PART3`.

**Returns:**

An array of ISO 3166 two-letter country codes.

## getISOCountries

```
public static Set<String> getISOCountries(Locale.IsoCountryCode type)
```

Returns an unmodifiable `Set` of ISO3166 country codes for the specified type.

**Parameters:**

`type` - `Locale.IsoCountryCode` specified ISO code type.

**Returns:**

an unmodifiable `Set` of ISO3166 country codes for the specified type

**Throws:**

`NullPointerException` - if type is null

**Since:**

9

**See Also:**

`Locale.IsoCountryCode`

## getISOLanguages

```
public static String[] getISOLanguages()
```

Returns a list of all 2-letter language codes defined in ISO 639. Can be used to obtain Locales.

**Note:**

- ISO 639 is not a stable standard— some languages' codes have changed. The list this function returns includes both the new and the old codes for the languages whose codes have changed.

- The `Locale` class also supports language codes up to 8 characters in length. Therefore, the list returned by this method does not contain ALL valid codes that can be used to obtain Locales.

**Returns:**

An array of ISO 639 two-letter language codes.

## getLanguage

```
public String getLanguage()
```

Returns the language code of this Locale.

**Implementation Note:**

This method returns the new forms for the obsolete ISO 639 codes ("iw", "ji", and "in"). See Legacy language codes for more information.

**Returns:**

The language code, or the empty string if none is defined.

**See Also:**

`getDisplayLanguage()`

## getScript

```
public String getScript()
```

Returns the script for this locale, which should either be the empty string or an ISO 15924 4-letter script code. The first letter is uppercase and the rest are lowercase, for example, 'Latn', 'Cyrl'.

**Returns:**

The script code, or the empty string if none is defined.

**Since:**

1.7

**See Also:**

`getDisplayScript()`

## getCountry

```
public String getCountry()
```

Returns the country/region code for this locale, which should either be the empty string, an uppercase ISO 3166 2-letter code, or a UN M.49 3-digit code.

**Returns:**

The country/region code, or the empty string if none is defined.

**See Also:**

`getDisplayCountry()`

## getVariant

```
public String getVariant()
```

Returns the variant code for this locale.

**Returns:**

The variant code, or the empty string if none is defined.

**See Also:**

`getDisplayVariant()`

## hasExtensions

```
public boolean hasExtensions()
```

Returns `true` if this `Locale` has any extensions.

**Returns:**

`true` if this `Locale` has any extensions

**Since:**

1.8

## stripExtensions

```
public Locale stripExtensions()
```

Returns a copy of this `Locale` with no extensions. If this `Locale` has no extensions, this `Locale` is returned.

**Returns:**

a copy of this `Locale` with no extensions, or `this` if `this` has no extensions

**Since:**

1.8

## getExtension

```
public String getExtension(char key)
```

Returns the extension (or private use) value associated with the specified key, or null if there is no extension associated with the key. To be well-formed, the key must be one of `[0-9A-Za-z]`. Keys are case-insensitive, so for example 'z' and 'Z' represent the same extension.

**Parameters:**

key - the extension key

**Returns:**

The extension, or null if this locale defines no extension for the specified key.

**Throws:**

`IllegalArgumentException` - if key is not well-formed

**Since:**

1.7

**See Also:**

`PRIVATE_USE_EXTENSION`, `UNICODE_LOCALE_EXTENSION`

## getExtensionKeys

```
public Set<Character> getExtensionKeys()
```

Returns the set of extension keys associated with this locale, or the empty set if it has no extensions. The returned set is unmodifiable. The keys will all be lower-case.

**Returns:**

The set of extension keys, or the empty set if this locale has no extensions.

**Since:**

1.7

## getUnicodeLocaleAttributes

```
public Set<String> getUnicodeLocaleAttributes()
```

Returns the set of unicode locale attributes associated with this locale, or the empty set if it has no attributes. The returned set is unmodifiable.

**Returns:**

The set of attributes.

**Since:**

1.7

## getUnicodeLocaleType

```
public String getUnicodeLocaleType(String key)
```

Returns the Unicode locale type associated with the specified Unicode locale key for this locale. Returns the empty string for keys that are defined with no type. Returns null if the key is not defined. Keys are case-insensitive. The key

must be two alphanumeric characters ([0-9a-zA-Z]), or an IllegalArgumentException is thrown.

**Parameters:**

key - the Unicode locale key

**Returns:**

The Unicode locale type associated with the key, or null if the locale does not define the key.

**Throws:**

`IllegalArgumentException` - if the key is not well-formed

`NullPointerException` - if key is null

**Since:**

1.7

## getUnicodeLocaleKeys

```
public Set<String> getUnicodeLocaleKeys()
```

Returns the set of Unicode locale keys defined by this locale, or the empty set if this locale has none. The returned set is immutable. Keys are all lower case.

**Returns:**

The set of Unicode locale keys, or the empty set if this locale has no Unicode locale keywords.

**Since:**

1.7

## toString

```
public final String toString()
```

Returns a string representation of this `Locale` object, consisting of language, country, variant, script, and extensions as below:

> language + "_" + country + "_" + (variant + "_#" | "#") + script + "_" + extensions

Language is always lower case, country is always upper case, script is always title case, and extensions are always lower case. Extensions and private use subtags will be in canonical order as explained in `toLanguageTag()`.

When the locale has neither script nor extensions, the result is the same as in Java 6 and prior.

If both the language and country fields are missing, this function will return the empty string, even if the variant, script, or extensions field is present (a

locale with just a variant is not allowed, the variant must accompany a well-formed language or country code).

If script or extensions are present and variant is missing, no underscore is added before the "#".

This behavior is designed to support debugging and to be compatible with previous uses of `toString` that expected language, country, and variant fields only. To represent a Locale as a String for interchange purposes, use `toLanguageTag()`.

Examples:

- `en`
- `de_DE`
- `_GB`
- `en_US_WIN`
- `de__POSIX`
- `zh_CN_#Hans`
- `zh_TW_#Hant_x-java`
- `th_TH_TH_#u-nu-thai`

**Overrides:**

`toString` in class `Object`

**Returns:**

A string representation of the Locale, for debugging.

**See Also:**

`getDisplayName()`, `toLanguageTag()`

## toLanguageTag

```
public String toLanguageTag()
```

Returns a well-formed IETF BCP 47 language tag representing this locale.

If this `Locale` has a language, country, or variant that does not satisfy the IETF BCP 47 language tag syntax requirements, this method handles these fields as described below:

**Language:** If language is empty, or not well-formed (for example "a" or "e2"), it will be emitted as "und" (Undetermined).

**Country:** If country is not well-formed (for example "12" or "USA"), it will be omitted.

**Variant:** If variant **is** well-formed, each sub-segment (delimited by '-' or '_') is emitted as a subtag. Otherwise:

- if all sub-segments match `[0-9a-zA-Z]{1,8}` (for example "WIN" or "Oracle_JDK_Standard_Edition"), the first ill-formed sub-segment and all following will be appended to the private use subtag. The first appended subtag will be "lvariant", followed by the sub-segments in order, separated by hyphen. For example, "x-lvariant-WIN", "Oracle-x-lvariant-JDK-Standard-Edition".
- if any sub-segment does not match `[0-9a-zA-Z]{1,8}`, the variant will be truncated and the problematic sub-segment and all following sub-segments will be omitted. If the remainder is non-empty, it will be emitted as a private use subtag as above (even if the remainder turns out to be well-formed). For example, "Solaris_isjustthecoolestthing" is emitted as "x-lvariant-Solaris", not as "solaris".

**Special Conversions:** Java supports some old locale representations, including deprecated ISO language codes, for compatibility. This method performs the following conversions:

- Deprecated ISO language codes "iw", "ji", and "in" are converted to "he", "yi", and "id", respectively.
- A locale with language "no", country "NO", and variant "NY", representing Norwegian Nynorsk (Norway), is converted to a language tag "nn-NO".

**Note:** Although the language tag obtained by this method is well-formed (satisfies the syntax requirements defined by the IETF BCP 47 specification), it is not necessarily a valid BCP 47 language tag. For example,

```
Locale.forLanguageTag("xx-YY").toLanguageTag();
```

will return "xx-YY", but the language subtag "xx" and the region subtag "YY" are invalid because they are not registered in the IANA Language Subtag Registry.

**Returns:**

a BCP47 language tag representing the locale

**Since:**

1.7

**See Also:**

`forLanguageTag(String)`

### caseFoldLanguageTag

```
public static String caseFoldLanguageTag(String languageTag)
```

Returns a case folded IETF BCP 47 language tag.

This method formats a language tag into one with case convention that adheres to section 2.1.1. Formatting of Language Tags of RFC 5646. This format is defined as: *All subtags, including extension and private use subtags, use lowercase letters with two exceptions: two-letter and four-letter subtags that neither appear at the start of the tag nor occur after singletons. Such two-letter subtags are all uppercase (as in the tags "en-CA-x-ca" or "sgn-BE-FR") and four- letter subtags are titlecase (as in the tag "az-Latn-x-latn").* As legacy tags, (defined as "grandfathered" in RFC 5646) are not always well-formed, this method will simply case fold a legacy tag to match the exact case convention for the particular tag specified in the respective `Legacy tags` table.

**Special Exceptions**

To maintain consistency with `variant` which is case-sensitive, this method will neither
case fold variant subtags nor case fold private use subtags prefixed by `lvariant`.

For example,

```
String tag = "ja-kana-jp-x-lvariant-Oracle-JDK-Standard-Edition";
Locale.caseFoldLanguageTag(tag); // returns "ja-Kana-JP-x-lvariant-Oracle-JDK-St
String tag2 = "ja-kana-jp-x-Oracle-JDK-Standard-Edition";
Locale.caseFoldLanguageTag(tag2); // returns "ja-Kana-JP-x-oracle-jdk-standard-e
```

Excluding case folding, this method makes no modifications to the tag itself. Case convention of language tags does not carry meaning, and is simply recommended as it corresponds with various ISO standards, including: ISO639-1, ISO15924, and ISO3166-1.

As the formatting of the case convention is dependent on the positioning of certain subtags, callers of this method should ensure that the language tag is well-formed, (conforming to section 2.1. Syntax of RFC 5646).

**Parameters:**

`languageTag` - the IETF BCP 47 language tag.

**Returns:**

a case folded IETF BCP 47 language tag

**Throws:**

`IllformedLocaleException` - if `languageTag` is not well-formed

`NullPointerException` - if `languageTag` is `null`

**Since:**

21

**External Specifications**

RFC 5646: 2.1. Syntax,
RFC 5646: 2.1.1. Formatting of Language Tags

## forLanguageTag

```
public static Locale forLanguageTag(String languageTag)
```

Returns a locale for the specified IETF BCP 47 language tag string.

If the specified language tag contains any ill-formed subtags, the first such subtag and all following subtags are ignored. Compare to `Locale.Builder.setLanguageTag(String)` which throws an exception in this case.

Duplicate variants are accepted and included by the builder. However, duplicate extension singleton keys and their associated type are accepted but ignored. The same behavior applies to duplicate locale keys and attributes within a U extension. Note that subsequent subtags after the occurrence of a duplicate are not ignored.

The following **conversions** are performed:

- The language code "und" is mapped to language "".
- The language codes "iw", "ji", and "in" are mapped to "he", "yi", and "id" respectively. (This is the same canonicalization that's done in Locale's constructors.) See Legacy language codes for more information.
- The portion of a private use subtag prefixed by "lvariant", if any, is removed and appended to the variant field in the result locale (without case normalization). If it is then empty, the private use subtag is discarded:

```
    Locale loc;
    loc = Locale.forLanguageTag("en-US-x-lvariant-POSIX");
    loc.getVariant(); // returns "POSIX"
    loc.getExtension('x'); // returns null

    loc = Locale.forLanguageTag("de-POSIX-x-URP-lvariant-Abc-Def");
```

```
    loc.getVariant(); // returns "POSIX_Abc_Def"
    loc.getExtension('x'); // returns "urp"
```

- BCP 47 language tags permit up to three extlang subtags. However, the second and third extlang subtags are always ignored. As such, the first extlang subtag in `languageTag` is used as the language, and the primary language subtag and other extlang subtags are ignored. Language tags that exceed three extlang subtags are considered ill-formed starting at the offending extlang subtag.

```
    Locale.forLanguageTag("ar-aao").getLanguage(); // returns "aao"
    Locale.forLanguageTag("en-abc-def-us").toString(); // returns "abc_US"
    Locale.forLanguageTag("zh-yue-gan-cmn-czh-CN").toString();
    // returns "yue"; "czh" exceeds the extlang limit, and subsequent
    // subtags are considered ill-formed
```

- Case is normalized except for variant tags, which are left unchanged. Language is normalized to lower case, script to title case, country to upper case, and extensions to lower case.
- If, after processing, the locale would exactly match either ja_JP_JP or th_TH_TH with no extensions, the appropriate extensions are added as though the constructor had been called:

```
   Locale.forLanguageTag("ja-JP-x-lvariant-JP").toLanguageTag();
   // returns "ja-JP-u-ca-japanese-x-lvariant-JP"
   Locale.forLanguageTag("th-TH-x-lvariant-TH").toLanguageTag();
   // returns "th-TH-u-nu-thai-x-lvariant-TH"
```

This implements the 'Language-Tag' production of BCP47, and so supports legacy (regular and irregular, referred to as "Type: grandfathered" in BCP47) as well as private use language tags. Stand alone private use tags are represented as empty language and extension 'x-whatever', and legacy tags are converted to their canonical replacements where they exist.

Legacy tags with canonical replacements are as follows:

| legacy tag | modern replacement |
|---|---|
| art-lojban | jbo |
| i-ami | ami |
| i-bnn | bnn |
| i-hak | hak |
| i-klingon | tlh |
| i-lux | lb |

| legacy tag | modern replacement |
|---|---|
| i-navajo | nv |
| i-pwn | pwn |
| i-tao | tao |
| i-tay | tay |
| i-tsu | tsu |
| no-bok | nb |
| no-nyn | nn |
| sgn-BE-FR | sfb |
| sgn-BE-NL | vgt |
| sgn-CH-DE | sgg |
| zh-guoyu | cmn |
| zh-hakka | hak |
| zh-min-nan | nan |
| zh-xiang | hsn |

Legacy tags with no modern replacement will be converted as follows:

| legacy tag | converts to |
|---|---|
| cel-gaulish | xtg-x-cel-gaulish |
| en-GB-oed | en-GB-x-oed |
| i-default | en-x-i-default |
| i-enochian | und-x-i-enochian |
| i-mingo | see-x-i-mingo |
| zh-min | nan-x-zh-min |

For a list of all legacy tags, see the IANA Language Subtag Registry (search for "Type: grandfathered").

**Note**: there is no guarantee that `toLanguageTag` and `forLanguageTag` will round-trip.

**Parameters:**

`languageTag` - the language tag

**Returns:**

The locale that best represents the language tag.

**Throws:**

`NullPointerException` - if `languageTag` is `null`

**Since:**

1.7

**See Also:**

toLanguageTag(),
Locale.Builder.setLanguageTag(String)

## getISO3Language

```
public String getISO3Language()
                        throws MissingResourceException
```

Returns a three-letter abbreviation of this locale's language. If the language matches an ISO 639-1 two-letter code, the corresponding ISO 639-2/T three-letter lowercase code is returned. The ISO 639-2 language codes can be found on-line, see "Codes for the Representation of Names of Languages Part 2: Alpha-3 Code". If the locale specifies a three-letter language, the language is returned as is. If the locale does not specify a language the empty string is returned.

**Returns:**

a three-letter abbreviation of this locale's language

**Throws:**

MissingResourceException - Throws MissingResourceException if three-letter language abbreviation is not available for this locale.

## getISO3Country

```
public String getISO3Country()
                       throws MissingResourceException
```

Returns a three-letter abbreviation of this locale's country. If the country matches an ISO 3166-1 alpha-2 code, the corresponding ISO 3166-1 alpha-3 uppercase code is returned. If the locale doesn't specify a country, this will be the empty string.

The ISO 3166-1 codes can be found on-line.

**Returns:**

a three-letter abbreviation of this locale's country

**Throws:**

MissingResourceException - Throws MissingResourceException if the three-letter country abbreviation is not available for this locale.

## getDisplayLanguage

```
public String getDisplayLanguage()
```

Returns a name for `this` locale's language that is appropriate for display to the user. If possible, the name returned will be localized for the default

DISPLAY locale. For example, if the locale is fr_FR and the default DISPLAY locale is en_US, getDisplayLanguage() will return "French"; if the locale is en_US and the default DISPLAY locale is fr_FR, getDisplayLanguage() will return "anglais". If the name returned cannot be localized for the default DISPLAY locale, this function falls back on the English name, and uses the ISO code as a last-resort value. If the locale doesn't specify a language, this function returns the empty string.

**Returns:**

The name of the display language appropriate to the default DISPLAY locale.

## getDisplayLanguage

```
public String getDisplayLanguage(Locale inLocale)
```

Returns a name for this locale's language that is appropriate for display to the user. If possible, the name returned will be localized according to inLocale. For example, if the locale is fr_FR and inLocale is en_US, getDisplayLanguage() will return "French"; if the locale is en_US and inLocale is fr_FR, getDisplayLanguage() will return "anglais". If the name returned cannot be localized according to inLocale, this function falls back on the English name, and finally on the ISO code as a last-resort value. If the locale doesn't specify a language, this function returns the empty string.

**Parameters:**

inLocale - The locale in which to localize the display language.

**Returns:**

The name of the display language appropriate to the given locale.

**Throws:**

NullPointerException - if inLocale is null

## getDisplayScript

```
public String getDisplayScript()
```

Returns a name for this locale's script that is appropriate for display to the user. If possible, the name will be localized for the default DISPLAY locale. Returns the empty string if this locale doesn't specify a script code.

**Returns:**

The display name of the script code appropriate to the default DISPLAY locale.

**Since:**

1.7

## getDisplayScript

```
public String getDisplayScript(Locale inLocale)
```

Returns a name for this locale's script that is appropriate for display to the user. If possible, the name will be localized for the given locale. Returns the empty string if this locale doesn't specify a script code.

**Parameters:**

inLocale - The locale in which to localize the display script.

**Returns:**

The display name of the script code appropriate to the given locale.

**Throws:**

NullPointerException - if inLocale is null

**Since:**

1.7

## getDisplayCountry

```
public String getDisplayCountry()
```

Returns a name for this locale's country that is appropriate for display to the user. If possible, the name returned will be localized for the default DISPLAY locale. For example, if the locale is fr_FR and the default DISPLAY locale is en_US, getDisplayCountry() will return "France"; if the locale is en_US and the default DISPLAY locale is fr_FR, getDisplayCountry() will return "Etats-Unis". If the name returned cannot be localized for the default DISPLAY locale, this function falls back on the English name, and uses the ISO code as a last-resort value. If the locale doesn't specify a country, this function returns the empty string.

**Returns:**

The name of the country appropriate to the default DISPLAY locale.

## getDisplayCountry

```
public String getDisplayCountry(Locale inLocale)
```

Returns a name for this locale's country that is appropriate for display to the user. If possible, the name returned will be localized according to inLocale. For example, if the locale is fr_FR and inLocale is en_US, getDisplayCountry() will return "France"; if the locale is en_US and inLocale is fr_FR, getDisplayCountry() will return "Etats-Unis". If the name returned cannot be localized according to inLocale, this function falls back on the English name, and finally on the ISO code as a last-resort value. If the locale doesn't specify a country, this function returns the empty string.

**Parameters:**

`inLocale` - The locale in which to localize the display country.

**Returns:**

The name of the country appropriate to the given locale.

**Throws:**

`NullPointerException` - if `inLocale` is `null`

## getDisplayVariant

```
public String getDisplayVariant()
```

Returns a name for `this` locale's variant code that is appropriate for display to the user. If possible, the name will be localized for the default `DISPLAY` locale. If the locale doesn't specify a variant code, this function returns the empty string.

**Returns:**

The name of the display variant code appropriate to the default `DISPLAY` locale.

## getDisplayVariant

```
public String getDisplayVariant(Locale inLocale)
```

Returns a name for `this` locale's variant code that is appropriate for display to the user. If possible, the name will be localized for inLocale. If the locale doesn't specify a variant code, this function returns the empty string.

**Parameters:**

`inLocale` - The locale in which to localize the display variant code.

**Returns:**

The name of the display variant code appropriate to the given locale.

**Throws:**

`NullPointerException` - if `inLocale` is `null`

## getDisplayName

```
public String getDisplayName()
```

Returns a name for `this` locale that is appropriate for display to the user. This will be the values returned by getDisplayLanguage(), getDisplayScript(), getDisplayCountry(), getDisplayVariant() and optional Unicode extensions assembled into a single string. The non-empty values are used in order, with the second and subsequent names in parentheses. For example:

language (script, country, variant(, extension)*)
language (country(, extension)*)
language (variant(, extension)*)
script (country(, extension)*)
country (extension)*

depending on which fields are specified in the locale. The field separator in the above parentheses, denoted as a comma character, may be localized depending on the locale. If the language, script, country, and variant fields are all empty, this function returns the empty string.

**Returns:**

The display name appropriate to the default `DISPLAY` locale.

## getDisplayName

```
public String getDisplayName(Locale inLocale)
```

Returns a name for `this` locale that is appropriate for display to the user. This will be the values returned by getDisplayLanguage(), getDisplayScript(), getDisplayCountry(), getDisplayVariant(), and optional Unicode extensions assembled into a single string. The non-empty values are used in order, with the second and subsequent names in parentheses. For example:

language (script, country, variant(, extension)*)
language (country(, extension)*)
language (variant(, extension)*)
script (country(, extension)*)
country (extension)*

depending on which fields are specified in the locale. The field separator in the above parentheses, denoted as a comma character, may be localized depending on the locale. If the language, script, country, and variant fields are all empty, this function returns the empty string.

**Parameters:**

`inLocale` - The locale in which to localize the display name.

**Returns:**

The display name appropriate to the given locale.

**Throws:**

`NullPointerException` - if `inLocale` is `null`

## clone

```
public Object clone()
```

Overrides Cloneable.

**Overrides:**

`clone` in class `Object`

**Returns:**

a clone of this instance.

**See Also:**

`Cloneable`

## hashCode

```
public int hashCode()
```

Override hashCode. Since Locales are often used in hashtables, caches the value for speed.

**Overrides:**

`hashCode` in class `Object`

**Returns:**

a hash code value for this object

**See Also:**

`Object.equals(java.lang.Object)`,
`System.identityHashCode(Object)`

## equals

```
public boolean equals(Object obj)
```

Returns true if this Locale is equal to another object. A Locale is deemed equal to another Locale with identical language, script, country, variant and extensions, and unequal to all other objects.

**Overrides:**

`equals` in class `Object`

**Parameters:**

`obj` - the reference object with which to compare.

**Returns:**

true if this Locale is equal to the specified object.

**See Also:**

`Object.hashCode()`, `HashMap`

## filter

```
public static List<Locale> filter
(List<Locale.LanguageRange> priorityList,
 Collection<Locale> locales,
 Locale.FilteringMode mode)
```

Returns a list of matching `Locale` instances using the filtering mechanism defined in RFC 4647. This filter operation on the given `locales` ensures that only unique matching locale(s) are returned.

**Parameters:**

`priorityList` - user's Language Priority List in which each language tag is sorted in descending order based on priority or weight

`locales` - `Locale` instances used for matching

`mode` - filtering mode

**Returns:**

a list of `Locale` instances for matching language tags sorted in descending order based on priority or weight, or an empty list if nothing matches. The list is modifiable.

**Throws:**

`NullPointerException` - if `priorityList` or `locales` are `null`. `NullPointerException` may be thrown if any elements within either `Collection` are `null`.

`IllegalArgumentException` - if one or more extended language ranges are included in the given list when `Locale.FilteringMode.REJECT_EXTENDED_RANGES` is specified

**Since:**

1.8

## filter

```
public static List<Locale> filter
(List<Locale.LanguageRange> priorityList,
 Collection<Locale> locales)
```

Returns a list of matching `Locale` instances using the filtering mechanism defined in RFC 4647. This is equivalent to `filter(List, Collection, FilteringMode)` when `mode` is `Locale.FilteringMode.AUTOSELECT_FILTERING`. This filter operation on the given `locales` ensures that only unique matching locale(s) are returned.

**Parameters:**

`priorityList` - user's Language Priority List in which each language tag is sorted in descending order based on priority or weight

`locales` - `Locale` instances used for matching

**Returns:**

a list of `Locale` instances for matching language tags sorted in descending order based on priority or weight, or an empty list if nothing matches. The list is modifiable.

**Throws:**

`NullPointerException` - if `priorityList` or `locales` are `null`. `NullPointerException` may be thrown if any elements within either `Collection` are `null`.

**Since:**

1.8

## filterTags

```
public static List<String> filterTags
(List<Locale.LanguageRange> priorityList,
 Collection<String> tags,
 Locale.FilteringMode mode)
```

Returns a list of matching languages tags using the basic filtering mechanism defined in RFC 4647. This filter operation on the given `tags` ensures that only unique matching tag(s) are returned with preserved case. In case of duplicate matching tags with the case difference, the first matching tag with preserved case is returned. For example, "de-ch" is returned out of the duplicate matching tags "de-ch" and "de-CH", if "de-ch" is checked first for matching in the given `tags`. Note that if the given `tags` is an unordered `Collection`, the returned matching tag out of duplicate tags is subject to change, depending on the implementation of the `Collection`.

**Parameters:**

`priorityList` - user's Language Priority List in which each language tag is sorted in descending order based on priority or weight

`tags` - language tags

`mode` - filtering mode

**Returns:**

a list of matching language tags sorted in descending order based on priority or weight, or an empty list if nothing matches. The list is modifiable.

**Throws:**

NullPointerException - if priorityList or tags are null. NullPointerException may be thrown if any elements within either Collection are null.

IllegalArgumentException - if one or more extended language ranges are included in the given list when Locale.FilteringMode.REJECT_EXTENDED_RANGES is specified

**Since:**

1.8

## filterTags

```
public static List<String> filterTags
(List<Locale.LanguageRange> priorityList,
 Collection<String> tags)
```

Returns a list of matching languages tags using the basic filtering mechanism defined in RFC 4647. This is equivalent to filterTags(List, Collection, FilteringMode) when mode is Locale.FilteringMode.AUTOSELECT_FILTERING. This filter operation on the given tags ensures that only unique matching tag(s) are returned with preserved case. In case of duplicate matching tags with the case difference, the first matching tag with preserved case is returned. For example, "de-ch" is returned out of the duplicate matching tags "de-ch" and "de-CH", if "de-ch" is checked first for matching in the given tags. Note that if the given tags is an unordered Collection, the returned matching tag out of duplicate tags is subject to change, depending on the implementation of the Collection.

**Parameters:**

priorityList - user's Language Priority List in which each language tag is sorted in descending order based on priority or weight

tags - language tags

**Returns:**

a list of matching language tags sorted in descending order based on priority or weight, or an empty list if nothing matches. The list is modifiable.

**Throws:**

NullPointerException - if priorityList or tags are null. NullPointerException may be thrown if any elements within either Collection are null.

**Since:**

1.8

## lookup

```
public static Locale lookup(List<Locale.LanguageRange> priorityList,
                            Collection<Locale> locales)
```

Returns a `Locale` instance for the best-matching language tag using the lookup mechanism defined in RFC 4647.

**Parameters:**

`priorityList` - user's Language Priority List in which each language tag is sorted in descending order based on priority or weight

`locales` - `Locale` instances used for matching

**Returns:**

the best matching `Locale` instance chosen based on priority or weight, or `null` if nothing matches.

**Throws:**

`NullPointerException` - if `priorityList` or `locales` are `null`. `NullPointerException` may be thrown if any elements within either `Collection` are `null`.

**Since:**

1.8

## lookupTag

```
public static String lookupTag
(List<Locale.LanguageRange> priorityList,
 Collection<String> tags)
```

Returns the best-matching language tag using the lookup mechanism defined in RFC 4647. This lookup operation on the given `tags` ensures that the first matching tag with preserved case is returned.

**Parameters:**

`priorityList` - user's Language Priority List in which each language tag is sorted in descending order based on priority or weight

`tags` - language tags used for matching

**Returns:**

the best matching language tag chosen based on priority or weight, or `null` if nothing matches.

**Throws:**

`NullPointerException` - if `priorityList` or `tags` are `null`. `NullPointerException` may be thrown if any elements within either `Collection` are `null`.

**Since:**

1.8

---

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.